



Synapse Optics

LensHH-LT

GPU Pre-Screen Architecture

Value-only Multistart sieve, CPU analytic LM polish

Release

LensHH-LT 1.0.115

June 4, 2026

A map for this deck — three flowcharts

Three flowcharts from the LensHH-LT user guide do the heavy lifting in this talk.

- Multistart, CPU-only — the baseline Basin-Hopping loop every release before 1.0.115 ran
- Inside one trial: HJ-LM — what each Multistart trial actually does on the CPU
- Multistart with GPU pre-screen — what changes in 1.0.115
- We walk the three diagrams first, then unpack the GPU mental model: one CUDA thread per candidate lens

Basin Hopping is the bottleneck of lens design

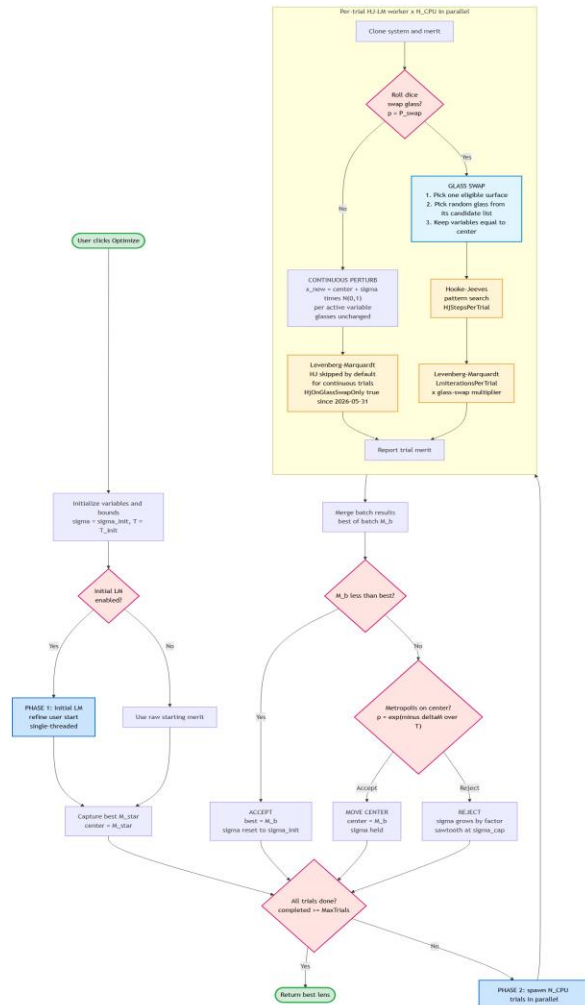
Local LM converges in seconds — lens design is a global problem.

- The wall-clock budget of a real design session is dominated by Basin Hopping (Multistart + Metropolis acceptance) — the exploration of design space
- Every accepted basin can re-anchor the search; every rejected basin still cost a full HJ-LM polish on the CPU
- The fundamental design choice (Cooke vs double-Gauss vs split-flint) is made during exploration. Everything downstream is refinement
- Speeding up Basin Hopping = speeding up the actual day-to-day work of a lens designer

Multistart today: the CPU-only baseline

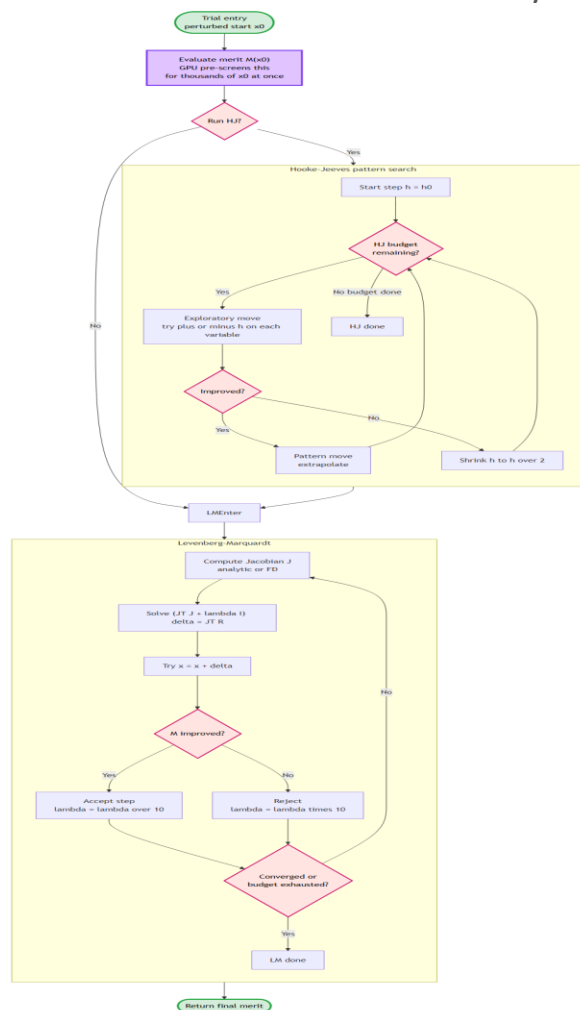
Same outer Basin-Hopping loop every release before 1.0.115.

- Phase 1: initial LM refines the user start
- Phase 2: spawn N_{CPU} trials in parallel — every trial is a full HJ-LM polish
- Each trial is expensive: a typical 20-surface multi-field polish ~ 80 ms times LM iterations times N_{CPU} cores
- Merge, accept best-of-batch or run Metropolis, then loop
- Throughput is bounded by HJ-LM cost times core count — the number of basins you can try in a fixed wall-clock



Inside one trial: HJ-LM

What each Multistart trial actually does on the CPU.

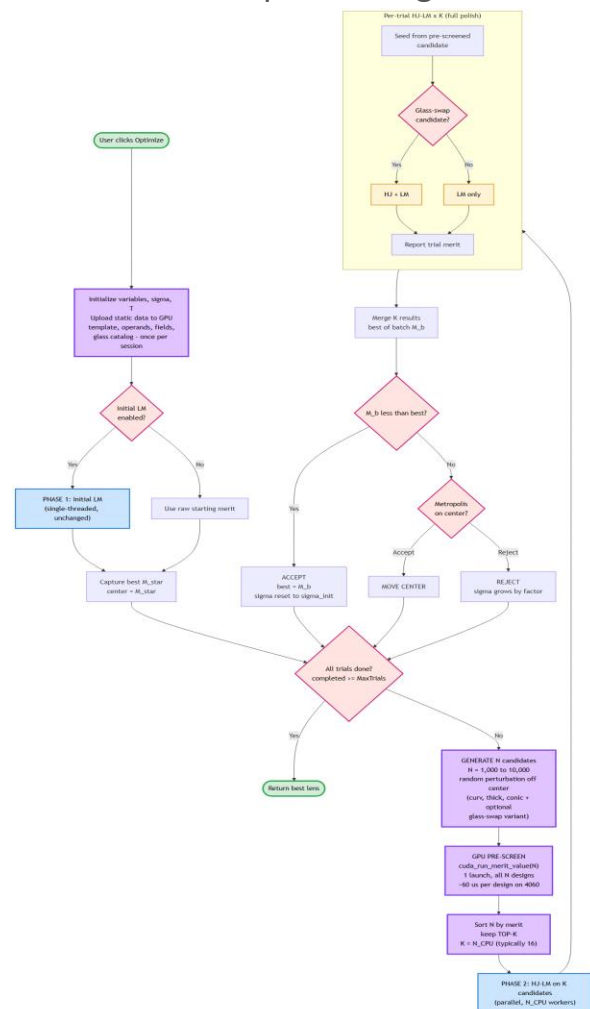


- Each trial clones the system, rolls a glass-swap dice, then perturbs continuous variables
- Glass-swap trials get Hooke-Jeeves pattern search PLUS Levenberg-Marquardt (HJ helps escape sharp local minima from the swap)
- Continuous trials run LM only — HJ is skipped by default since 2026-05-31
- LM with analytical Jacobian (1.0.115) drops the per-iteration cost $\sim 4x$ vs FD — that win compounds across every trial

Multistart with GPU pre-screen — 1.0.115

Same outer loop. One stage added between candidate generation and HJ-LM polish.

- GENERATE N = 1,000 to 10,000 candidates — much wider than the CPU could ever polish
- GPU PRE-SCREEN: one CUDA launch, full merit value per candidate, ~60 us per design on a 4060
- Sort by merit, keep top-K (K = N_CPU, typically 16 to 64) — the rest are abandoned
- CPU HJ-LM polish runs only on the survivors — same cost as before, applied to better candidates



How it looks in 1.0.115

The GPU pre-screen toggle in the Multistart dialog — Beta-labeled, auto-disabled when the design uses features the GPU kernel does not cover.

Multi Start Optimization

Hardware acceleration ☒ Use GPU pre-screen (Beta)

CUDA device detected. When enabled, each Phase-2 batch generates ~16× more candidate designs than CPU cores, evaluates all on GPU in one launch (~60 μs/design on 4060), and runs HJ-LM only on the top survivors. Continuous variables AND glass-swap candidates are sieved together.

Advanced (engine, derivative, LM internals) ▾

Trials: 2000 LM/Trial: 4000 Init LM: 200 Init Sigma: 0.001 Sigma Cap: 0.5

Glass Sub %: 50 ☐ Constrained Only ☒ Brodyen Update ☒ Metropolis Acceptance HJ Steps: 10

Init Damp: 0.001 ☐ Skip Init LM

Ready 0.0 s Accepted: 0

Initial: Post-LM: Current: Best:

Variable	Start	Best	Delta	Sur	Start Glas	Best Glass

When enabled, each Phase-2 trial batch generates ~16× more candidate designs than CPU cores and evaluates them all on the GPU in one launch (60 μs per design on a 4060). Only the top-N survivors run the expensive HJ-LM polish. Currently shipping in 1.0.115 as a detection-only Beta; the inner-loop integration lands in 1.0.116.

Start Stop OK — Accept Results Cancel — Revert

- Engine / Derivative controls sit alongside the Hardware acceleration strip
- GPU pre-screen toggle defaults ON when a CUDA device is detected and the active design is GPU-eligible
- Auto-disables with a visible reason when it cannot help (aspherics today, certain multi-config layouts)
- Same dialog, same workflow as before — pre-screen is an additive option, never a behavior change

The shift: wider exploration in the same wall-clock

Pre-screen cost is $\sim 1000\times$ cheaper per candidate than HJ-LM polish.

- Pre-screen ~ 60 us per candidate on GPU vs HJ-LM ~ 80 ms per candidate on CPU
- For the same wall-clock that CPU-only spent polishing K candidates, 1.0.115 can pre-screen 1,000K candidates AND polish the K best
- CPU-only Basin Hopping was always picking the best-of- K random shots; 1.0.115 picks the best-of-1000K random shots, then polishes those
- The exploration radius around the current center grows because we sample more before committing budget
- Same Metropolis acceptance, same convergence guarantees — just a wider net cast at each step

Every lens is a thread

The CUDA mental model behind the pre-screen kernel.

- CUDA grid: `gridDim = (N candidates)`, one thread per candidate design
- Each thread loads only its own variables — `curvature[V]`, `thickness[V]`, `conic[V]`, `material_ids[N_surfaces]`
- All threads execute the SAME code: the whole `evaluate_merit` value path
- Session-persistent device data shared by all threads: glass catalog, fields, operands, merit template
- Thread divergence is data-driven, not code-driven — same instructions, different lens

How big is the batch, and how fast does it run?

Each lens is a thread. The per-SM concurrent thread budget is FUNCTION-DEPENDENT — set by register and shared-memory pressure per thread.

- `evaluate_merit` is register-heavy (ray trace + refraction + operand scratch per thread): 256 concurrent threads per SM is the practical budget — well below the hardware's theoretical 1,536-2,048
- Resident-thread budget per launch = $N_SMs \times 256$:
- RTX 4060 (consumer): $24 \times 256 = \sim 6,100$ resident lenses (one wave)
- A100 80GB (data center): $108 \times 256 = \sim 27,600$ resident lenses — about 4.5x wider wave than 4060
- H100 SXM5 (data center): $132 \times 256 = \sim 33,800$ resident lenses — about 5.5x wider wave than 4060
- Launching more candidates than one wave is fine — extra warps queue and run as later waves; the SM never idles. The 256 number is wave width, not the launch cap.
- Throughput follows the total FP64 TFLOPS ratio. 4060 = 0.24 TFLOPS, A100 = 9.7 TFLOPS, H100 = 34 TFLOPS. `evaluate_merit` is FP64-compute-bound (sin / cos / sqrt / division per ray, state lives in registers), so wall-clock scales close to the TFLOPS ratio — memory bandwidth ($\sim 7x$ A100, $\sim 12x$ H100) sets a floor but does not bind here.
- A100 vs 4060: $\sim 40x$ throughput — process $\sim 40x$ more lenses per second, or finish the same batch in $\sim 1/40$ the time. H100 vs 4060: $\sim 140x$ throughput.

Why filling GPU occupancy matters

Memory latency is hidden by parallel work, not by faster cores.

- Global memory access takes ~400 to 800 cycles to return on a GPU — much longer than a CPU cache miss
- A stalled thread is invisible if the SM can swap in another warp (32 threads) that is ready to run
- Each SM needs MANY resident warps to keep its execution units fed — that is the definition of "occupancy". At 256 concurrent threads per SM we get 8 warps in flight per SM — enough to overlap typical memory latency
- Under-occupied launches (a few candidates per SM, not a few thousand total) leave SMs idle on memory stalls — wasted silicon, no speedup vs CPU
- Multistart's natural width — 1,000 to 10,000+ random candidates per outer iteration — saturates a consumer 4060's ~6K-thread budget and starts to make the A100's ~28K-thread budget worth paying for

What runs on the device

Whole evaluate_merit value path, ported to __device__ via LENSHH_HD header refactor.

- About 1,560 lines of merit-eval code — paraxial, refraction, surface intersection, ray trace, robust ray-aim, exit-pupil trace, every operand family
- Bit-equal to the CPU value path by construction — same source, just compiled for the device
- Validation: representative imaging lenses (Cooke triplet at 6 surfaces, complex multi-element designs at 20+ surfaces) both produce $\delta = 0$ vs CPU evaluate_merit
- No derivatives, no Duals, no Jacobian — pure scalar value path

Glass swap is a single int

Discrete glass substitutions ride alongside continuous perturbations in the same launch.

- Device-side glass catalog stored once per session: $K_{\text{glasses}} \times W_{\text{wavelengths}}$ refractive-index table
- Each candidate carries a `material_ids[N_surfaces]` int32 array — one int per surface
- Swapping N-BK7 for N-SK16 on surface 3 changes ONE int — no host->device transfer, no re-upload
- A single batched launch can mix continuous candidates with glass-swap variants
- Combinatorial coverage of geometry x glass that single-trial CPU never had the budget for

Why value-only? Why not GPU Jacobian?

Use each piece of silicon where it wins.

- Consumer GPU FP64 throughput is the binding constraint: ~0.23 TFLOPS on a 4060
- CPU AVX2 FP64 is comparable or better for sequential dual-number derivative work — about 0.8 TFLOPS
- CPU analytic Jacobian on a typical multi-field merit is already 13 ms (4x faster than FD)
- GPU comparative advantage is WIDTH — thousands of independent candidates, not narrow derivative chains
- GPU for pre-screen breadth, CPU for analytic LM depth

GPU FP64 landscape — consumer to data center

Why "value-only" is the right architectural choice for consumer GPUs — and what scales on A100 / H100.

Model	SMs	FP32 (TFLOPS)	FP64 (TFLOPS)	Memory	Bandwidth
RTX 4060 (desktop)	24	15.1	0.24	8 GB GDDR6	0.272 TB/s
A100 80GB	108	19.5	9.7 vector / 19.5 tensor	80 GB HBM2e	2.0 TB/s
H100 SXM5	132	67	34 vector / 67 tensor	80 GB HBM3	3.35 TB/s
H200 SXM	132	67	34 vector / 67 tensor	141 GB HBM3e	4.8 TB/s
B200	148	70-80	40 tensor	192 GB HBM3e	8 TB/s
B300	148	high	1.4 (FP64 gutted)	288 GB HBM3e	8 TB/s

Consumer 4060 FP64 ~0.24 TFLOPS, data-center A100 vector FP64 ~9.7 TFLOPS — a ~40x ceiling lift. Note Blackwell B300 explicitly gutted FP64 for AI workloads.

FP64 per dollar — what each tier actually costs

Throughput-per-dollar drives the deployment decision. Cloud rental is where the real lever is.

Model	Vector FP64	Buy (USD)	FP64 / \$1k	Cloud \$/hr	FP64 per \$/hr
RTX 4060	0.24	300	0.80	0.15	1.6
A100 80GB	9.7	10,000	0.97	1.50	6.5
H100 SXM5	34	28,000	1.21	2.50	13.6
H200 SXM	34	32,000	1.06	3.70	9.2
B200	20 (est)	35,000	0.57	5.50	3.6
B300	1.4 (est)	40,000+	0.035	n/a	n/a

Cloud H100 at \$2.50/hr buys the most FP64 per dollar-hour. Buying a 4060 for a developer desk still gives surprisingly good FP64 per dollar.

Measured throughput

Where 1.0.115 lands on real fixtures.

- Consumer 4060 (24 SMs, sm_89): about 2x end-to-end Multistart speedup across a representative range of imaging lenses (6 to 20+ surfaces)
- Per-design value cost ~60 us — thousands of candidates per second on commodity hardware
- CPU pieces (analytic LM, hybrid CT-family / TTRACK / span ops) keep their existing speed unchanged
- A100 (data-center): code path is implemented end-to-end and expected to scale with FP64 throughput; cloud-test validation pending
- Combined with the analytic-Jacobian + oversubscription fixes, 1.0.115 is the biggest end-to-end Multistart speedup the project has shipped

Summary

One mental model, three numbers.

- Basin Hopping (Multistart + Metropolis) dominates the wall-clock of a real lens-design session
- Every lens is a thread — N candidate designs evaluated in one CUDA launch
- GPU runs the whole merit function VALUE — no derivatives, no Jacobian
- CPU does the analytical Jacobian during HJ-LM polish on the top-K survivors
- Net effect: same Basin-Hopping loop, but the exploration radius widens by $\sim 1000\times$ for the same wall-clock